

HYBRID CONTEXTUAL MARKET BASKET ANALYTICS

Midway Progress Report | Big Data Analytics



Objective

Design and implement a hybrid Big Data pipeline that combines historical Market Basket Analysis with real-time clickstream behavior to issue a targeted coupon when a shopper shows purchase hesitation on a strongly associated product. This report records the current vertical slice, the course concepts demonstrated, and the remaining distributed-data stages.

Status	Midway vertical slice is runnable
Current live path	Randomized simulator -> Kafka -> stateful coupon engine -> Redis activity log -> Flask SSE viewer
Rule artifact	14,344 CSV rows, 77 representative products; provisional and explicitly not the final Spark result
Evidence	Kafka/Redis Docker stack, live event feed, issued and suppressed decisions, automated policy tests
Report date	20 September 2026

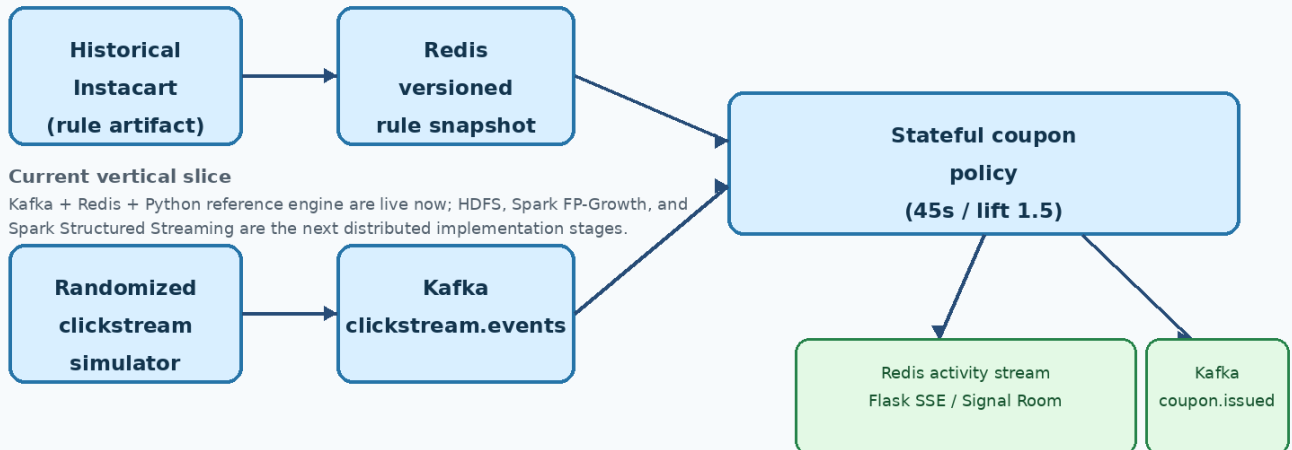
Repository evidence

Branch: feat/live-randomized-stream-viewer | Commit: f8381d5 | Pull request:
github.com/f-society67/MBA_Analysis/pull/1

1. Application of Big Data Analytics Concepts

The implemented first vertical slice applies distributed-system concepts at the boundaries where they matter: Kafka provides an ordered event log, Redis provides low-latency rule and observation lookups, and a stateful policy consumes the stream by shopper/session. The current Python engine is intentionally a reference implementation for the future Spark Structured Streaming job.

Hybrid batch + streaming coupon pipeline



Decision rule demonstrated

For each session, the engine tracks cart state and product views. It issues a 5% coupon only when `dwelt_seconds >= 45`, the viewed product is not already in the cart, a directed cart-item -> viewed-product rule has lift ≥ 1.5 , and that shopper/product has not already received an offer. The strongest qualifying cart item is selected.

Code excerpt: stateful hesitation policy

```
dwelt = float(event["dwelt_seconds"])
if dwelt < self.dwelt_threshold_seconds:
    return None, "below_dwelt_threshold"
if product in state.cart:
    return None, "product_in_cart"
for cart_item in state.cart:
    metrics = self.rules.lookup(cart_item, product)
    if metrics and float(metrics["lift"]) >= self.min_lift:
        best = max(best, (float(metrics["lift"]), cart_item, metrics))
```

Streaming contract

Events are JSON objects with `event_id`, `event_type`, `event_time`, `user_id`, `session_id`, and `product_id`. View-ended events add `dwelt_seconds`; cart events add `quantity`. The Kafka events topic is partitioned by `user_id` so state mutations for one shopper remain ordered. Coupon output includes support, confidence, lift, discount, and the supporting cart item.

2. Project Progress

Progress is separated into delivered work and planned work so the midpoint claim remains technically honest. The live track is usable today against the provisional rule snapshot; the historical distributed track is the next major implementation milestone.

Workstream	State	Evidence / boundary
Repository and environment	Complete	uv project, Docker Compose, Kafka 4.0.2 KRaft, Redis 7.4, repeatable scripts.
Data and rule artifact	Partial	14,344 rows and 77 representative products loaded; old notebook flattening defect remains to be replaced by Spark FP-Growth.
Event contract	Complete	Validation for four event types, ISO-8601 UTC time, cart quantities, and dwell values.
Batch analytics	Planned	HDFS Parquet landing and Spark MLlib FP-Growth are specified but not yet implemented.
Streaming decision loop	Complete reference	Kafka consumer, Redis lookup, stateful policy, coupon.issued output, bounded observer stream.
Randomized feed	Complete	Real product/rule identifiers sampled with eligible, short-view, already-in-cart, and no-match scenarios.
Viewer and tests	Complete	Flask Signal Room with SSE, counters, evidence card; seven policy/simulator tests pass.

Runtime verification captured for this report

Check	Observed result
Viewer endpoint	HTTP 200 at http://127.0.0.1:5000/viewer
Event feed	3,189 observed events in Redis activity counters
Policy branches	516 issued and 401 suppressed decisions
Automated tests	7 passed (engine thresholds, in-cart guard, lift guard, idempotency, reasons, random catalog branches)

The screenshot-like evidence panel above is generated from the same viewer API used by the UI; it is a runtime snapshot, not a fabricated benchmark.

3. Methods, Tools, and APIs

The stack is intentionally staged. The implemented components prove the event and serving contracts before the heavier HDFS/Spark deployment is introduced. This avoids building a distributed job around an untested rule schema.

Tool / API	Role now	Why appropriate / next use
Apache Kafka 4.0.2 (KRaft)	clickstream.events and coupon.issued topics	Durable, partitioned event log; KRaft removes ZooKeeper dependency. The same contract will feed Spark Structured Streaming.
Redis 7.4	rule snapshot + bounded activity stream + counters	Fast hash lookups suit a hot rule cache and low-latency observability. Future snapshots should be versioned and published by Spark.
Python reference engine	session state, cart logic, dwell/lift policy	Small, testable behavioral oracle for the future distributed stream job; not presented as Spark execution.
Flask + SSE	/viewer, /api/activity, /api/activity/stream	Minimal operations surface that renders live decisions without polling; supports Last-Event-ID reconnects.
Docker Compose	local Kafka and Redis orchestration	Reproducible isolated course demonstration; later extend with Spark/HDFS containers or a cluster profile.
Pandas / mlxtend / scikit-learn	legacy notebook and provisional CSV artifact	Useful for EDA/prototyping, but the current flattened rule export is not final evidence. Replace with Spark MLlib FP-Growth.
uv + pytest	dependency lock and 7 automated tests	Fast reproducible setup and regression safety around event contracts and coupon policy.

Randomization and replay

pipeline/live_simulator.py reads actual product names and directed qualifying pairs from app/rules.csv, weights products by available support, and samples four scenario branches: eligible (55%), short_view (20%), already_in_cart (15%), and no_match (10%). The --seed option makes an experiment replayable; the default feed is intentionally different from run to run.

Code excerpt: non-hardcoded product feed

```
scenario = rng.choices(list(SCENARIO_WEIGHTS),
weights=list(SCENARIO_WEIGHTS.values()), k=1)[0]
pair = catalog.choose_pair(rng)
```

This makes the live result meaningful for the current demo: the viewer sees varied cart/view products and both coupon outcomes, while all source identifiers are traceable to the local rule artifact.

4. Team Participation and Equal Work Distribution

The team is distributing the project as three equal ownership tracks. Each member owns one third of the delivery surface, with explicit handoffs at the versioned rule snapshot, event contract, and acceptance tests. The split below is an equal-weight plan for the delivered vertical slice and its immediate continuation; it does not imply that the unbuilt Spark stages are already complete.

Member	Equal share	Ownership and contribution	Handoff / acceptance
Vaibhav P	33.33% (1/3)	Platform and integration: Docker Compose, Kafka topics, Redis rule loading, event contract, consumer orchestration, Flask routes, SSE plumbing, run scripts, branch/PR integration.	Kafka/Redis services start; event schema is validated; viewer and scripts are reachable.
Antony Johnson	33.33% (1/3)	Historical intelligence: Instacart data interpretation, support/confidence/lift semantics, provisional-rule audit, Spark FP-Growth migration design, HDFS/Parquet landing and rule-quality plan.	One-item directional rule schema is documented; Spark output will be validated before Redis publication.
Harish M	33.33% (1/3)	Activation and quality: randomized simulator, scenario coverage, stateful coupon policy, Redis observability stream, Signal Room UI, policy tests, runtime demonstration and documentation.	Eligible and suppression branches are visible; tests assert thresholds, guards, idempotency, and random products.

Collaboration flow

Antony Johnson Historical track	<-- versioned rules + metrics -->	Vaibhav P Platform track	<-- event contract + APIs -->	Harish M Activation track
HDFS / Spark FP-Growth	shared acceptance	Kafka / Redis / Flask	shared acceptance	engine / simulator / UI

Collaboration evidence

The current branch is `feat/live-randomized-stream-viewer`, with the implementation committed as `f8381d5` and published in the open pull request at github.com/f-society67/MBA_Analysis/pull/1. The report itself is generated from the repository so a reviewer can inspect the exact code, tests, scripts, and runtime API described here.

5. Explanation of Work Completed and Next Stage

The project now has an end-to-end demonstration loop. A randomized shopper session adds one or more products to a cart, starts and ends a product view, and publishes JSON events to Kafka. The consumer validates each event, updates per-session state, checks the Redis rule snapshot, and emits either a coupon.issued event or an explicit suppression reason. The same observation is written to a bounded Redis stream. Flask exposes the stream over server-sent events, and the Signal Room renders counters, an event tape, and the evidence behind the latest decision.

How to run the demonstration

1. Install locked dependencies: `uv sync`. 2. Start the continuous stack: `bash scripts/run_live.sh`. 3. Open `http://127.0.0.1:5000/viewer`. 4. Watch varied products, dwell values, issued coupons, and suppressed branches. 5. Press Ctrl-C to stop, or set `KEEP_INFRA=1` to retain Kafka and Redis for inspection. For a finite terminal scenario, use `bash scripts/run_demo.sh`.

Reproducibility commands

```
bash scripts/run_live.sh
# browser: http://127.0.0.1:5000/viewer
uv run pytest -q
# expected: 7 passed
```

Remaining loose ends / next stage

Priority	Deliverable	Definition of done
1	HDFS landing	Pin the Instacart source version; validate all six tables; write immutable, partitioned Parquet and document schema/lineage.
2	Spark MLlib FP-Growth	Mine frequent itemsets at scale; preserve arrays or explicitly filter one-item pairs; publish quality metrics and a versioned Redis snapshot.
3	Spark Structured Streaming	Replace the Python reference consumer with checkpoints, watermark/state policy, Kafka offsets, Redis lookup, and measured latency/throughput.
4	Product/evaluation loop	Expose server-approved offers in the storefront; replay scenarios; measure precision, suppression reasons, latency, and coupon guardrails. No causal uplift claim yet.

Midway conclusion

The project has crossed the most important integration threshold: a continuously pumping, randomized stream produces observable, stateful actions in a viewer. The next phase should replace the provisional batch artifact with a validated HDFS + Spark pipeline, then port the proven decision contract to Spark Structured Streaming. That sequence keeps the demonstration honest while preserving a clear path to the full Big Data architecture.